Демидовский университет

Рисование и анимации

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Система рисования

- Система рисования Qt позволяет рисовать на экране, изображениях и виджетах, используя один и тот же API
- QPainter используется для выполнения операций рисования
  - QPaintDevice — это абстракция двумерного пространства, на котором можно рисовать с помощью QPainter. Класс скрыт
- QPaintEngine предоставляет интерфейс, который QPainter использует для рисования на различных элементах
  - От него наследуются QImage, QPicture, QPixmap, QSvgGenerator, QWidget и другие

https://doc.qt.io/qt-6/paintsystem.html

Демидовский университет

Рисование и заполнение

- QPainter предоставляет высокооптимизированные функции для выполнения большинства программ рисования
- Он может рисовать простые графические примитивы (QPoint, QLine, QRect, QRegion и QPolygon) и сложные фигуры (векторные пути)
  - Векторные пути представлены классом QPainterPath
- Линии и контуры рисуются с помощью класса QPen
- Кисть пера — это объект QBrush, используемый для заливки штрихов, созданных с помощью пера
- QPainter может рисовать текст и пиксельные изображения
  - Для текста шрифт указывается с помощью QFont

https://doc.qt.io/qt-6/paintsystem-drawing.html

Демидовский университет

Рисование и заполнение

- Формы заполняются с помощью класса QBrush
- Любой цвет в Qt представлен классом QColor, который поддерживает цветовые модели RGB, HSV и CMYK
- Доступные узоры заливки описываются перечислением Qt::BrushStyle. К ним относятся базовые узоры, включающие в себя однородные цвета, очень разреженные узоры, различные комбинации линий, градиентные заливки и текстуры
- QGradient определяет пользовательские градиентные заливки
- Узоры текстур указываются с помощью класса QPixmap

https://doc.qt.io/qt-6/paintsystem-drawing.html

Демидовский университет

Система координат

- Система координат управляется классом QPainter
- Система координат по умолчанию на объекте для рисования имеет начало в верхнем левом углу. Единицей по умолчанию является один пиксель на пиксельных устройствах и одна точка (1/72 дюйма) на принтерах
- Сопоставление логических координат QPainter с физическими координатами QPaintDevice обрабатывается матрицей преобразования QPainter, областью просмотра и "окном"
  - Логическая и физическая системы координат по умолчанию совпадают
- QPainter поддерживает преобразования координат (например, поворот и масштабирование)

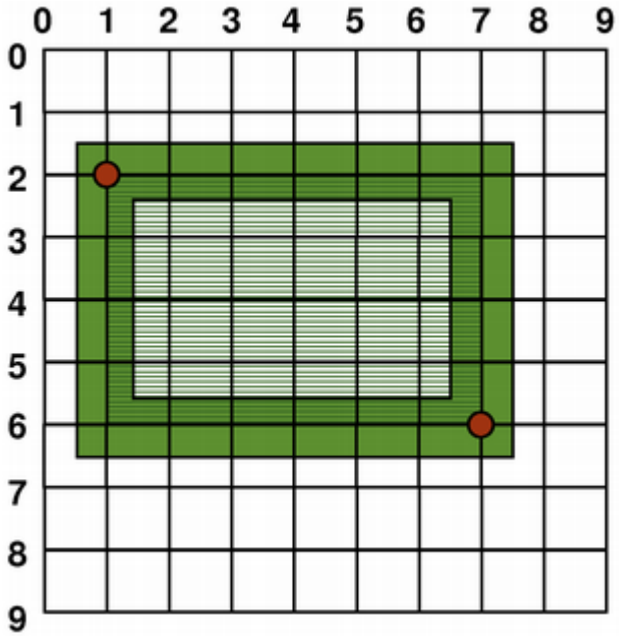
Демидовский университет

https://doc.qt.io/qt-6/coordsys.html

Размеры объектов

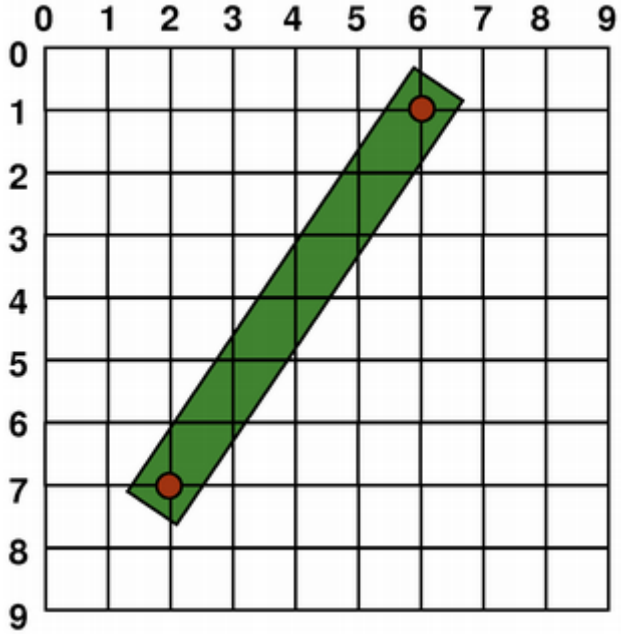
- Размер (ширина и высота) графического примитива всегда соответствует его математической модели, независимо от ширины пера, которым он визуализируется

https://doc.qt.io/qt-6/coordsys.html



QRect(1, 2, 6, 4)

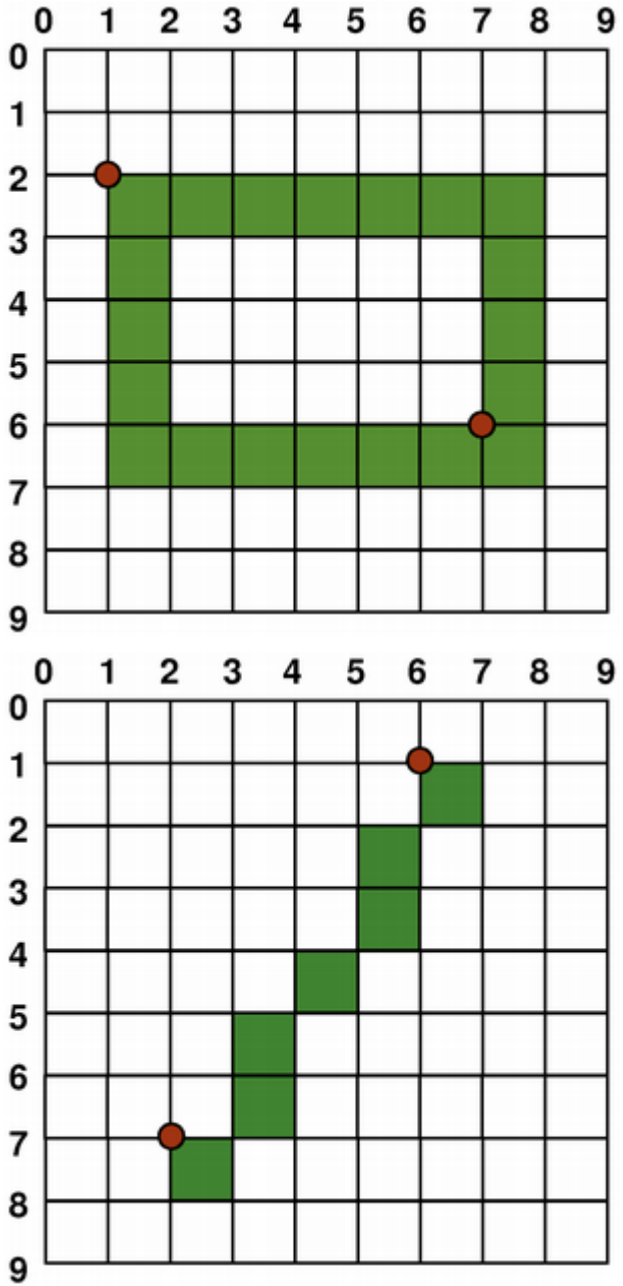
QLine(2, 7, 6, 1)



Сглаживание

- Рендеринг пикселей контролируется подсказкой QPainter.RenderHint
- QPainter.Antialiasing указывает, что движок должен сглаживать края примитивов, если это возможно, используя различную интенсивность цвета
- По умолчанию при рендеринге с помощью пера шириной в один пиксель пиксели будут отображаться справа и ниже математически опредёленных точек

https://doc.qt.io/qt-6/coordsys.html



QPainter – инструмент для рисования

- paintEvent у QWidget – событие рисования виджета
  - В нем происходит работа с QPainter
- drawPoint(), drawPoints(), drawLine(), drawRect(), drawRoundedRect(), drawEllipse(), drawArc(), drawPie(), drawChord(), drawPolyline(), drawPolygon(), drawConvexPolygon(), drawCubicBezier() – рисование примитивов
- setPen(), setBrush() – настроить инструменты рисования
- fillRect(), fillPath() – заполнить внутреннюю часть фигуры при работе с путём
- strokePath() – отрисовать границу при работе с путем в целом

https://doc.qt.io/qtforpython-6/PySide6/QtGui/QPainter.html

QPen – карандаш для рисования

- Рисует линию
- setStyle() – стиль линии (сплошная, пунктир, …)
- setWidth() – ширина линии
- setBrush() – настройка кисти, часто задается просто как цвет
- setCapStyle() – сглаживание линии
- setJoinStyle() – стиль соединения нескольких линий
- Стиль часто задается как набор констант, в том числе из перечислений

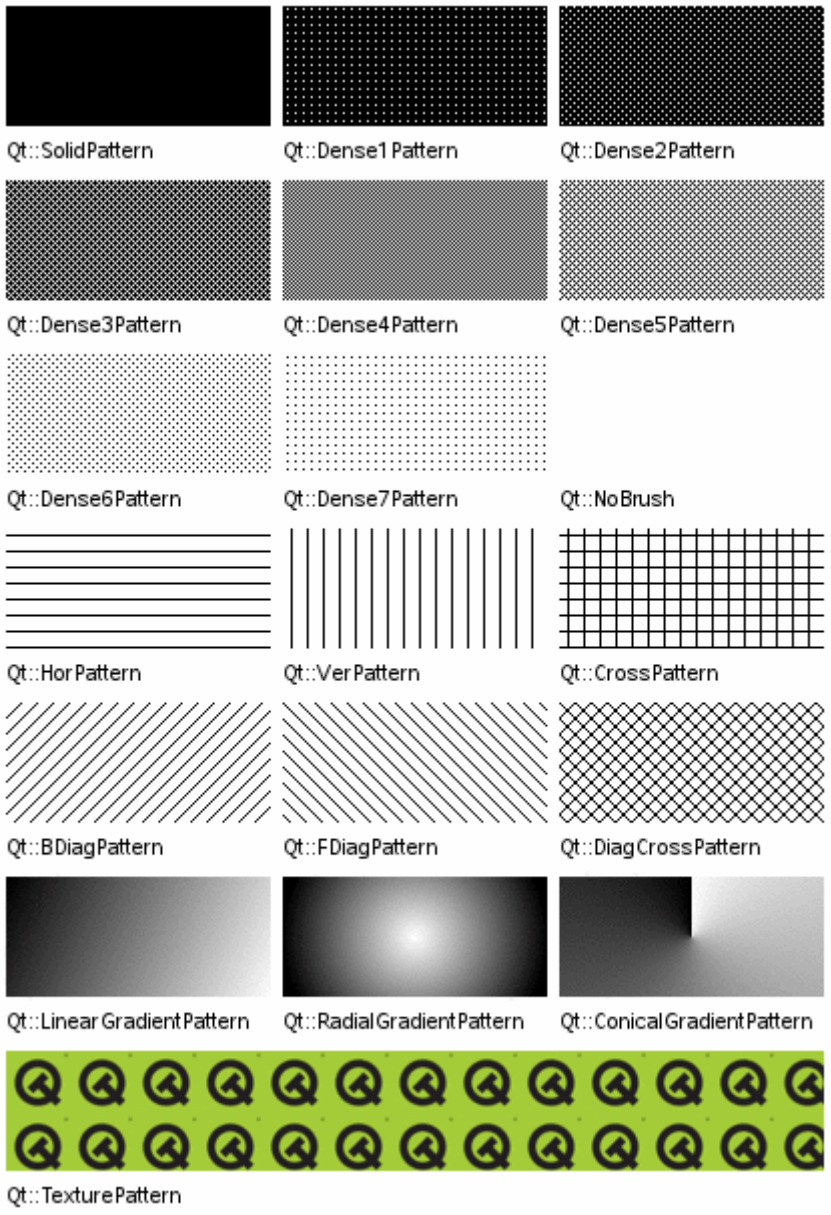
https://doc.qt.io/qtforpython-6/PySide6/QtGui/QPen.html

Демидовский университет

QBrush – кисть для рисования

- Заполняет пространство в заданном стиле
- setStyle() – стиль кисти
- setColor() – цвет кисти

https://doc.qt.io/qtforpython-6/PySide6/QtGui/QBrush.html



Градиенты

- QGradient – базовый класс для градиентов
- QLinearGradient – линейные градиенты интерполируют цвета между начальной и конечной точками
- QRadialGradient – радиальные градиенты интерполируют цвета между фокусной точкой и конечными точками окружности, окружающей ее
- QConicalGradient – конические градиенты интерполируют цвета вокруг центральной точки

https://doc.qt.io/qtforpython-6/PySide6/QtGui/QGradient.html

Демидовский университет

Определение градиентов

- QGradient – setColorAt()/setStops() задают точки градиента
- QLinearGradient – setStart() и setFinalStop() задают начало и конец градиента
- QRadialGradient – setCenter() и setRadius() задают центр и радиус круга градиента
- QConicalGradient – setAngle() и setCenter() задают угол и центральную точку для градиента

Демидовский университет

Пример простого рисования

```
def paintEvent(self, arg__0):
    painter = QPainter(self)
    painter.setPen(Qt.green)
    painter.drawLine(QLineF(50.0, 10.0, 10.0, 90.0))
    painter.drawLine(QLineF(50.0, 10.0, 90.0, 90.0))
    painter.drawLine(QLineF(10.0, 90.0, 90.0, 90.0))
    pen = QPen()
    pen.setStyle(Qt.DashDotLine)
    pen.setWidth(3)
    pen.setBrush(Qt.red)
    painter.setPen(pen)
    radial_gradient = QRadialGradient(100, 100, 50)
    radial_gradient.setColorAt(0.0, Qt.white)
    radial_gradient.setColorAt(0.2, Qt.green)
    radial_gradient.setColorAt(1.0, Qt.black)
    painter.setBrush(QBrush(radial_gradient))
    painter.drawRect(90, 90, 150, 150)
    painter.setPen(Qt.blue)
    painter.drawText(self.rect(), Qt.AlignCenter, "Qt")
```

Демидовский университет

QPainterPath – контейнер для операций рисования

- Контейнер состоит из графических строительных блоков, таких как прямоугольники, эллипсы, линии и кривые
- Строительные блоки можно объединять в замкнутые пути, например, в виде прямоугольника или эллипса
- Замкнутый путь имеет совпадающие начальную и конечную точки
- Или они могут существовать независимо как незамкнутые подпути, такие как линии и кривые
- QPainterPath можно использовать для заполнения, выделения и обрезки

https://doc.qt.io/qtforpython-6/PySide6/QtGui/QPainterPath.html

Демидовский университет

QPainterPath – операции рисования

- addEllipse(), addPath(), addPolygon(), addRect(), addRegion(), addRoundedRect(), addText() – добавить элемент в путь
- moveTo() – сместиться по координатам
- arcTo(), cubicTo(), lineTo(), quadTo() – нарисовать линию
- painter.drawPath(path) – для рисования пути нужен QPainter

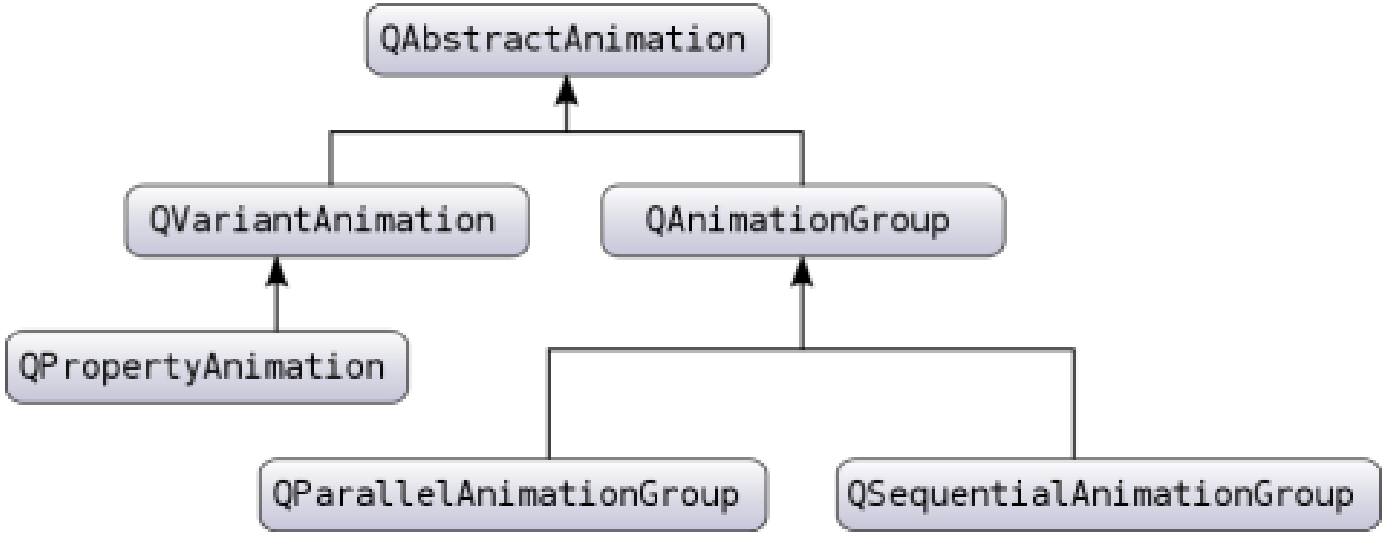
Демидовский университет

Пример QPainterPath

```
def paintEvent(self, arg__0):
    path = QPainterPath()
    path.addRect(20, 20, 60, 60)
    path.moveTo(0, 0)
    path.cubicTo(99, 0, 50, 50, 99, 99)
    path.cubicTo(0, 99, 50, 50, 0, 0)
    painter = QPainter(self)
    painter.fillRect(0, 0, 100, 100, Qt.white)
    painter.setPen(QPen(QColor(79, 106, 25), 1, Qt.SolidLine, Qt.FlatCap, Qt.MiterJoin))
    painter.setBrush(QColor(122, 163, 39))
    painter.drawPath(path)
    myPath = QPainterPath()
    myPath.addText(QPointF(150, 150), QFont("Times", 24, QFont.Bold), "Qt")
    gradient = QLinearGradient(QPointF(150, 150), QPointF(200, 200))
    gradient.setColorAt(0.0, Qt.yellow)
    gradient.setColorAt(1.0, Qt.green)
    painter.setBrush(gradient)
    painter.setPen(QPen())
    painter.drawPath(myPath)
```

Демидовский университет

Анимации виджетов



https://doc.qt.io/qtforpython-6/overviews/animation-overview.html

Демидовский университет

QAbstractAnimation – общие свойства анимаций

- setDuration() – продолжительность анимации
- start() , stop() , pause() , resume() – действия с анимацией
- setLoopCount() – сколько раз проиграть анимацию
- setDirection() – направление анимации вперёд или назад
- stateChanged() – изменилось состояние, например, анимацию поставили на паузу
- finished() – анимация завершилась

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QAbstractAnimation.html

Демидовский университет

QVariantAnimation – настройка линии для анимации

- setStartValue(), setEndValue() – задать начальное и конечное значение для анимируемого свойства
- setKeyValueAt() – задать значение посередине
- setKeyValues() – задать траекторию списком
- setEasingCurve() – задать кривую для анимации
  - QEasingCurve.Linear – прямая линия
  - QEasingCurve.InQuad, QEasingCurve.OutQuad – ветвь параболы
  - QEasingCurve.InOutQuad, QEasingCurve.OutInQuad – ветви двух парабол
  - …

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QVariantAnimation.html

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QEasingCurve.html

QPropertyAnimation – анмация свойства

- setPropertyName() – свойство, которое анимируется
- setTargetObject() – у какого виджета анимируется свойство

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QPropertyAnimation.html

Демидовский университет

Пример анимации свойств

```
class Window(QWidget):
    def __init__(self):
        super().__init__()
        self.setMinimumWidth(1000)
        self.setMinimumHeight(500)
        self.setWindowTitle("Анимация виджетов")
        button = QPushButton('Я еду по экспоненте', self)
        anim = QPropertyAnimation(button, b"pos", self)
        anim.setDuration(5000)
        anim.setStartValue(QPoint(0, 0))
        anim.setEndValue(QPoint(100, 250))
        anim.setEasingCurve(QEasingCurve.InExpo)
        anim.start()
```

Демидовский университет

Группировка анимаций

- QSequentialAnimationGroup – последовательные анимации
- QParallelAnimationGroup – параллельные анимации
  - addAnimation() – добавить анимацию в группу
  - start() – начать анимацию

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QSequentialAnimationGroup.html

https://doc.qt.io/qtforpython-6/PySide6/QtCore/QParallelAnimationGroup.html

Пример группировки анимаций

```
class Window(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Последовательная анимация виджетов")
        firstButton = QPushButton('Сначала еду я', self)
        firstAnim = QPropertyAnimation(firstButton, b"pos", self)
        firstAnim.setDuration(1000)
        firstAnim.setStartValue(QPoint(0, 0))
        firstAnim.setEndValue(QPoint(100, 250))
        secondButton = QPushButton('А потом еду я', self)
        secondButton.move(QPoint(100, 250))
        secondAnim = QPropertyAnimation(secondButton, b"pos", self)
        secondAnim.setDuration(3000)
        secondAnim.setStartValue(QPoint(100, 250))
        secondAnim.setEndValue(QPoint(500, 500))
        sequenceAnim = QSequentialAnimationGroup(self)
        sequenceAnim.addAnimation(firstAnim)
        sequenceAnim.addAnimation(secondAnim)
        sequenceAnim.start()
```